

AWS Foundations for Beginners

Cloud Basics · AWS Console · IAM · CDK · ShopFlow Architecture · Free Tier

AWS Tutorial · ShopFlow

01-00

What is AWS — And Why Should I Care?

Amazon Web Services (AWS) is a platform that rents you computing resources — servers, databases, storage, networking — over the internet, by the minute. Instead of buying physical servers (expensive, slow, one-time capacity), you rent exactly what you need and pay only for what you use. ShopFlow is an e-commerce platform we will build entirely on AWS, step by step, throughout this tutorial series.

■ Cloud Computing

Renting computers and services over the internet instead of buying and running your own hardware.

Example: Instead of buying a \$5,000 server for your office, you rent an AWS server for \$0.02/hour and only pay when it is running.

Without AWS (Traditional)	With AWS
Buy servers upfront (\$50,000+)	Rent servers by the hour (\$0.02-\$3.00/hr)
Takes weeks to provision	Takes seconds to launch
Pay for 100% capacity even at 2% usage	Pay only for what you actually use
You manage hardware failures	AWS replaces failed hardware automatically
Hard to scale for traffic spikes (Black Friday)	Scale from 10 to 10,000 servers in minutes
One data centre = one point of failure	Multiple regions worldwide; built-in redundancy

01-01

ShopFlow — The App We Will Build

Throughout this tutorial series, every AWS concept is taught in the context of ShopFlow — a real-world e-commerce platform. By the end of all 20 chapters, you will have designed and deployed a production-grade system capable of serving 1 million monthly users and handling 10,000 requests per second during Black Friday.

ShopFlow — What the App Does

Customers browse products, add items to cart, and place orders

Sellers list products with images, descriptions, and prices

Payments are processed securely (Stripe integration)

Orders are tracked from placement to delivery

Real-time search: find products by name, category, and price range

Analytics: live dashboards showing sales, popular products, revenue

ShopFlow Scale (what we are designing for)	Target
Monthly active users (MAU)	1,000,000 users

ShopFlow Scale (what we are designing for)	Target
Normal traffic	1,000 requests/second
Black Friday peak traffic	10,000 requests/second (10x normal)
Order data retention	7 years (financial compliance)
Product images stored	~500 GB, growing 10 GB/month
Target availability	99.99% uptime (52 minutes downtime/year maximum)
Target cost	~\$32,000/month at full scale

01-02

Creating Your AWS Account

Every AWS resource you create lives inside an AWS Account. Think of it as your "workspace" in the cloud. You need a credit card to create an account, but AWS has a Free Tier that lets you use many services at no cost for 12 months. We will set up billing alerts in section 01-06 so you never get a surprise bill.

■ Create a new AWS Account

1. Go to <https://aws.amazon.com> and click Create an AWS Account (top right)
2. Enter your email address and choose a strong password — this is your root account
3. Enter your account name (e.g. "ShopFlow Tutorial")
4. Enter payment information — you will only be charged if you exceed Free Tier limits
5. Verify your phone number via SMS
6. Choose Support Plan → select Basic support (free)
7. Sign in at <https://console.aws.amazon.com> — you are in!

■■ Never use the root account for daily work

The root account has unlimited permissions and cannot be restricted. After creating it, immediately create an IAM user (section 01-07) and use that for everything. Store the root password somewhere safe and only use it for account-level changes.

■ aws > AWS Sign In

Sign in to the AWS Management Console

Root user email address

your@email.com

■ Enter the email you used when creating the account

■ For daily work, sign in as an IAM user, not root

■ AWS will ask for MFA code if Multi-Factor Authentication is enabled (recommended)

Next

01-03

Navigating the AWS Console

The AWS Management Console is your graphical control panel for all AWS services. When you first log in it looks overwhelming — there are 200+ services! Do not worry; you will only use about 15 for ShopFlow. Here is a guided tour of the console layout.

AWS Management Console

■ Top Navigation Bar

- **Services** ■ — click to search and navigate to any AWS service
- **Search box** — fastest way to find a service: type "EC2", "RDS", "S3"
- **Region selector** (top right) — shows which AWS region you are in (e.g. us-east-1)
- **Account name** ■ (top right) — billing, account settings, sign out

■ Console Home — Recently Visited

■ AWS shows your recently used services here. At first it is empty.

■ Searching for a Service — the fastest method

Search bar (press / to focus)

■ Start typing a service name

Search results

■ EC2 — Virtual Servers in the Cloud ■ EC2 Image Builder ■ EC2 Auto Scaling

Console Navigation Method	How to Use It	Best For
Top search bar (press /)	Type service name (e.g. "S3", "Lambda", "RDS")	Finding any service quickly — fastest method
Services ■ dropdown	Browse by category (Compute, Storage, Database...)	Exploring what AWS offers in a category
Favorites (■ pin a service)	Click ■ next to a service to pin it to the top bar	Services you use every day (EC2, S3, CloudWatch)
Recent history	Automatically shows last 3 visited services	Quickly returning to where you were

01-04

AWS Regions and Availability Zones

AWS operates in 34 geographic Regions worldwide (as of 2024). Each Region is a cluster of data centres in one city or area. Inside each Region there are 2-6 Availability Zones (AZs) — physically separate buildings with independent power, cooling, and networking. This is how AWS achieves high availability: if one building has a power outage, your app keeps running in the other AZs.

■ AWS Region

A geographic area containing multiple AWS data centres. Examples: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

Example: ShopFlow uses us-east-1 (N. Virginia) as its primary region — closest to most North American users and cheapest for most AWS services.

■ Availability Zone (AZ)

A physically separate data centre building inside a Region. Each AZ has independent power and networking. If one AZ fails, others continue running.

Example: ShopFlow uses 3 AZs: us-east-1a, us-east-1b, us-east-1c. ShopFlow web servers run in all three — an outage in 1a still leaves 2 running.

Change your AWS Region

■ How to switch to us-east-1 (N. Virginia)

■ Look at the top-right of the console — you will see a region name like "Ohio" or "N. Virginia"

Click the region name to open the dropdown and select:

■ US East (N. Virginia) us-east-1 ■ US East (Ohio) us-east-2 ■ US West (Oregon) us-west-2

■ For this tutorial, always use us-east-1. Every resource you create is tied to the region you are currently viewing.

01-05

ShopFlow Services Map — Which AWS Service Does What

ShopFlow uses 15 AWS services. Here is a plain-English explanation of what each one does in the context of our app. Do not worry if these are unfamiliar — each service gets its own full chapter.

AWS Service	Plain-English Role in ShopFlow	Chapter
ECS Fargate	Runs our Java Spring Boot microservices (containers, no server management)	01
Aurora PostgreSQL	Main relational database: orders, users, products, payments	06
DynamoDB	NoSQL database: shopping cart, user sessions (ultra-fast reads/writes)	07
ElastiCache Redis	In-memory cache: product prices, popular product lists (millisecond reads)	08
S3	Stores product images and files; data lake for analytics	05
CloudFront	CDN: serves product images to users from the nearest edge location worldwide	05
API Gateway + Lambda	Serverless API endpoints: image resize, PDF receipt generation	09
Kinesis Data Streams	Real-time event pipeline: clickstream, order events, analytics	18
OpenSearch	Full-text product search: "red running shoes size 10"	12
SQS + SNS	Message queues: order confirmation emails, inventory updates	10
Cognito	User sign-up, sign-in, JWT authentication tokens	17
WAF	Web Application Firewall: block SQL injection, bot attacks	17
Route 53	DNS: shopflow.com points to our load balancer; failover routing	13
ALB	Application Load Balancer: distributes traffic across our 6+ servers	13
CloudWatch + X-Ray	Monitoring, logs, alerts, distributed tracing	14

01-06

Setting Up Billing Alerts — Do This First!

Before creating any AWS resources, set up a billing alert. This sends you an email if your AWS spend exceeds a threshold (e.g. \$10/month). Without this, you might accidentally leave a resource running and get an unexpected bill. Free Tier covers many services for 12 months, but not all, and limits exist.

■ Set Up Your First Billing Alert

1. In the AWS Console, click your account name (top right) → Billing and Cost Management
2. In the left sidebar, click Billing Preferences
3. Under Alert preferences, check ☒ Receive AWS Free Tier alerts (sends email as you approach Free Tier limits)
4. Check ☒ Receive CloudWatch billing alerts (required to create threshold alerts)
5. Click Save preferences
6. Now go to CloudWatch: search "CloudWatch" in the top bar and click it
7. In the left sidebar: Alarms → Billing → Create alarm
8. Set threshold: Estimated Charges > \$10 USD
9. Create SNS notification: enter your email address
10. Confirm the subscription email that AWS sends you
11. Done! You will receive an email if your monthly spend exceeds \$10

■ aws > CloudWatch > Alarms > Billing

Create billing alarm

■ Step 1 — Specify metric and conditions

Metric name

EstimatedCharges

■ Pre-filled; this tracks your total AWS bill

Currency

USD ■

■ Step 2 — Conditions

Threshold type

☒ Static ☒ Anomaly detection

Whenever EstimatedCharges is...

☒ Greater than ☒ Greater than or equal to

than... (USD)

10

■ Enter the dollar amount that triggers the alert

■ This alarm will trigger at the END of the month if your bill exceeds \$10

Next

01-07

Creating Your First IAM User — Stop Using Root!

IAM (Identity and Access Management) controls who can do what in your AWS account. The root account (your sign-up email) has unlimited power — including deleting your entire account. Never use it for daily work. Create an IAM user with Administrator access for your day-to-day tasks. This is the first security step every AWS engineer must do.

■ Create an IAM Admin User

1. Search "IAM" in the console and click Identity and Access Management (IAM)
2. In the left sidebar, click Users → Create user
3. Username: enter "shopflow-admin" (or your name)
4. Check ☒ Provide user access to the AWS Management Console
5. Select: I want to create an IAM user (not Identity Center)
6. Auto-generated password: check ☒ Users must create a new password at next sign-in
7. Click Next: Set permissions
8. Click Attach policies directly → search for "AdministratorAccess" → check the box ☒
9. Click Next → Create user
10. Download the CSV file with credentials (you only see this once!)
11. Sign in with the new IAM user URL shown on screen

■ aws > IAM > Users > Create user

Create user

■ Step 1 — Specify user details

User name

shopflow-admin

☒ Use lowercase letters, numbers, hyphens only

■ Step 2 — Console access

■ Provide user access to the AWS Management Console

Are you providing console access to a person?

☒ I want to create an IAM user ☐ Specify a user in Identity Center (recommended)

Console password

☒ Autogenerated password ☐ Custom password

☒ Users must create a new password at next sign-in (recommended)

■ After creating this user, bookmark the IAM sign-in URL and use it instead of the root login

Next

01-08

Installing the AWS CLI

The AWS CLI (Command Line Interface) lets you control AWS from your terminal. Instead of clicking through the console every time, you run commands like "aws s3 cp photo.jpg s3://my-bucket/" or "aws ec2 describe-instances". The CLI is essential for scripting and automation. CDK (next section) also uses the CLI under the hood.

■ Install and Configure AWS CLI (macOS/Linux)

1. Open your terminal
2. macOS: brew install awscli | Linux: sudo apt install awscli | Windows: download MSI from aws.amazon.com/cli
3. Verify: aws --version (should show aws-cli/2.x.x)
4. Configure: aws configure
5. Enter your IAM user Access Key ID (from the CSV you downloaded)
6. Enter your IAM user Secret Access Key
7. Default region: us-east-1
8. Default output format: json
9. Test: aws sts get-caller-identity (shows your account ID — you are connected!)

AWS CLI Command	What it Does	ShopFlow Example
aws configure	Set up credentials and default region	Run once after installing CLI
aws s3 ls	List S3 buckets in your account	aws s3 ls → shows shopflow-product-images
aws ec2 describe-instances	List running EC2/Fargate tasks	See what is running in your account
aws logs tail /aws/ecs/shopflow --follow	Stream live logs from ECS tasks	Debug a running ShopFlow service in real-time
aws cloudformation describe-stacks	List CDK-deployed stacks	See all ShopFlow infrastructure stacks

01-09

Introduction to AWS CDK — Infrastructure as Code

CDK (Cloud Development Kit) lets you define AWS infrastructure in code — TypeScript, Python, Java, or Go. Instead of clicking through the console every time (error-prone, hard to repeat), you write code that creates your infrastructure automatically. ShopFlow uses TypeScript CDK for all infrastructure. CDK generates CloudFormation templates which AWS uses to create resources.

■ Infrastructure as Code (IaC)

Defining your cloud infrastructure (servers, databases, load balancers) as code files, stored in Git, just like your application code.

Example: Instead of clicking "Create RDS database" in the console, you write: `new rds.DatabaseCluster(this, "DB", {...})` and run `cdk deploy`. Repeatable, versioned, reviewable.

How CDK Works — Step by Step

You write TypeScript code describing your AWS resources (CDK constructs)

`cdk synth` — CDK converts your code to a CloudFormation JSON template

`cdk deploy` — CDK uploads the template; CloudFormation creates the AWS resou

CloudFormation tracks what it created; `cdk diff` shows what changed

`cdk destroy` — removes all resources that CDK created (for cleanup)

Without CDK (Manual Console)

Click through 20 screens to create a VPC

With CDK

`cdk deploy` creates VPC in 2 minutes from code

Without CDK (Manual Console)	With CDK
Forget one security group setting — security hole	CDK code is reviewed in pull requests like app code
Recreating prod environment in staging takes days	cdk deploy in staging/prod — identical infrastructure
No history of who changed what infrastructure	Git history shows every infrastructure change
Hard to delete all resources (what did I create?)	cdk destroy removes everything CDK created

01-10

Your First CDK Project — Hello ShopFlow

Let us set up CDK and create your first AWS resource — an S3 bucket for product images. You will install CDK, initialize a project, write code to create the bucket, and deploy it to AWS. This is the foundation pattern for everything in ShopFlow.

■ Set Up CDK and Deploy Your First Resource

1. Install Node.js (required for CDK): <https://nodejs.org> → LTS version
2. Install CDK: `npm install -g aws-cdk`
3. Verify: `cdk --version` (should show 2.x.x)
4. Create project folder: `mkdir shopflow-infra && cd shopflow-infra`
5. Initialize CDK project: `cdk init app --language typescript`
6. CDK creates: `lib/shopflow-infra-stack.ts` (your main stack file)
7. Bootstrap CDK in your AWS account (one-time setup): `cdk bootstrap aws://YOUR-ACCOUNT-ID/us-east-1`
8. Open `lib/shopflow-infra-stack.ts` in your editor and add the S3 bucket code (see below)
9. Deploy: `cdk deploy` (takes 1-2 minutes)
10. Verify: go to S3 in the console — you should see your new bucket!

Your First CDK Code — S3 Bucket for Product Images

```
import * as cdk from "aws-cdk-lib";
import * as s3 from "aws-cdk-lib/aws-s3";
import { Construct } from "constructs";
export class ShopFlowInfraStack extends cdk.Stack {
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {
    super(scope, id, props);
    // Create an S3 bucket for ShopFlow product images
    const productImagesBucket = new s3.Bucket(this, "ProductImages", {
      bucketName: "shopflow-product-images-" + this.account,
      // Block all public access (images served via CloudFront, not directly)
      blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
      // Encrypt all images at rest
      encryption: s3.BucketEncryption.S3_MANAGED,
      // Keep old versions of images (safe deletion)
      versioned: true,
      // Remove bucket when you run cdk destroy (good for tutorials)
      removalPolicy: cdk.RemovalPolicy.DESTROY,
      autoDeleteObjects: true,
    });
    // Print the bucket name after deploy
    new cdk.CfnOutput(this, "BucketName", {
      value: productImagesBucket.bucketName,
    });
  }
}
```



```
}  
}
```

01-11

Verifying Your S3 Bucket in the Console

After running "cdk deploy", verify your S3 bucket was created correctly in the AWS Console. This step-by-step walkthrough shows what to look for.

aws > S3 > Buckets

Amazon S3 — Buckets

■ After cdk deploy, your bucket appears in the list

Search for buckets by name

shopflow-product-images

■ You should see: shopflow-product-images-123456789012 (your account ID appended)

■ Click on the bucket to verify settings

■ Properties tab → Bucket Versioning: Enabled

■ Properties tab → Default encryption: Amazon S3 managed key (SSE-S3)

■ Permissions tab → Block all public access: On (all four settings enabled)

■ Your first CDK-created resource is live! This bucket will store all ShopFlow product images.

■ CDK vs Console: When to use each

Use the console to VERIFY and INSPECT resources. Use CDK to CREATE and MODIFY resources. Never modify a CDK-managed resource directly in the console — your next cdk deploy will overwrite your manual change. The CDK code is the source of truth.

01-12

AWS Free Tier — What is Free and What is Not

AWS offers a Free Tier for new accounts (first 12 months). Many services are free up to certain limits. Understanding what is free helps you experiment without fear. For this tutorial, we will use Free Tier where possible. Production ShopFlow at scale is not free, but learning definitely can be.

AWS Service	Free Tier Limit (12 months)	What Happens After	ShopFlow Tutorial Impact
EC2	750 hours/month t2.micro or t3.micro	Billed at on-demand rate (~\$0.0116/hr)	Use t3.micro for any EC2 we create
S3	5 GB storage; 20,000 GET; 2,000 PUT requests	\$0.023/GB/month after 5 GB	Our product images bucket stays free for
RDS	750 hours/month db.t2.micro; 20 GB storage	\$0.017/hr after free tier	Use db.t3.micro Aurora for tutorial; delete
Lambda	1 million requests/month forever (always free)	\$0.000000166 per million requests	Lambda is effectively always free at tutorial
CloudWatch	10 custom metrics; 5 GB logs forever	\$0.30/GB after 5 GB	Set 7-day log retention to stay within limit
DynamoDB	25 GB storage; 25 WCU + 25 RCU forever	\$0.000065/WCU-hr after free tier	Free at tutorial scale (very low traffic)
ECR (Docker)	500 MB private + 50 GB public forever	\$0.10/GB/month for private	Pull public images; our tutorial images are

■ Avoid surprise bills during this tutorial

Always run "cdk destroy" at the end of each chapter to delete resources. Enable the billing alert from section 01-06. Use us-east-1 (cheapest region for most services). Never leave an RDS instance or NAT Gateway running overnight during learning.

A critical concept for every AWS engineer: AWS and you share responsibility for security. AWS secures the physical infrastructure (data centres, hardware, network). You are responsible for what you put IN the cloud (your data, your code, your configuration). This division is called the Shared Responsibility Model.

What AWS is Responsible For	What You Are Responsible For
Physical data centre security (guards, cameras, biometric access)	Your application code and its security vulnerabilities
Hardware (servers, network equipment, storage devices)	IAM: who has access to what in your account
Hypervisor (the software that runs virtual machines)	Encryption of your data (at rest and in transit)
AWS managed service uptime (e.g., S3, DynamoDB)	Security group rules (which ports are open to whom)
Patching the underlying infrastructure and managed services	Operating system patches on EC2 (for instances you manage)
Global network redundancy (multiple regions, AZs)	Backup and recovery of your application data

■ ShopFlow Security Responsibilities

For ShopFlow, you are responsible for: (1) IAM roles with least-privilege permissions, (2) encrypting all data at rest (Aurora, S3, DynamoDB, ElastiCache), (3) TLS/HTTPS for all traffic, (4) rotating secrets (database passwords, API keys) via Secrets Manager, (5) keeping your container images up to date (ECR scanning). AWS handles the rest.

Multi-Factor Authentication (MFA) adds a second layer of security: even if someone steals your password, they cannot log in without also having your phone. Enable MFA on your root account and your IAM admin user. Use an authenticator app (Google Authenticator, Authy) — not SMS which can be SIM-swapped.

■ Enable MFA on Your IAM User

1. Sign in as your IAM user (not root)
2. Click your username in the top right → Security credentials
3. Scroll to Multi-factor authentication (MFA) → Assign MFA device
4. Choose Authenticator app
5. Open Google Authenticator or Authy on your phone → Add account → Scan QR code
6. Enter two consecutive 6-digit codes from your authenticator app
7. Click Add MFA — done! Next login will ask for your MFA code

Assign MFA device

■ Choose MFA device type

MFA device type

■ Authenticator app (most secure) ■ Security Key (YubiKey etc) ■ Hardware TOTP token

■ Set up device — scan this QR code with your authenticator app

■ Open Google Authenticator → + → Scan QR code → point camera at the QR code shown in the console

MFA code 1

123456

■ Enter the first 6-digit code from your app

MFA code 2

789012

■ Wait 30 seconds and enter the next code

■ After this, every login will require: password + 6-digit code from your phone

Add MFA

Here is the complete architecture we will build across all 20 chapters. At this stage, just read it as a map — each service gets its own chapter with hands-on console walkthroughs and CDK code. By Chapter 20, every piece will be in place.

ShopFlow Complete AWS Architecture

Users → Route 53 DNS → CloudFront CDN → WAF

shopflow.com → ALB (Application Load Balancer)

Product images → S3 bucket (via CloudFront edge cache)

Application Layer (ECS Fargate — Docker containers)

order-svc: create orders, track status

product-svc: product catalog, search, recommendations

user-svc: sign-up, sign-in, profile (Cognito integration)

notification-svc: email/SMS for order confirmation (SQS + SNS)

Data Layer

Aurora PostgreSQL: orders, products, users (relational data)

DynamoDB: shopping cart, sessions (fast NoSQL)

ElastiCache Redis: product prices, popular lists (in-memory cache)

OpenSearch: full-text product search

Analytics and Events

Kinesis Data Streams → S3 Data Lake → Athena → QuickSight

01-16

Understanding AWS Support Plans

AWS offers four support plans. For this tutorial, Basic (free) is fine. For a production ShopFlow deployment, Business support (\$100/month minimum) is the minimum recommended — it gives you access to AWS engineers 24/7 and a Technical Account Manager to help with architecture questions.

Support Plan	Cost	Response Time	Best For
Basic (free)	\$0/month	AWS documentation and forums only; no live support	Learning and development; tutorials like this
Developer	\$29/month or 3% of bill	Business hours email support; 12-24 hour response	Small apps, testing, staging environments
Business	\$100/month or 3-10% of bill	24/7 phone, chat, email; 1-hour response for critical issues	Production workloads with real users (ShopFlow)
Enterprise On-Ramp	\$5,500/month	30-minute response for critical; dedicated TAM pool	High-traffic production systems; pre-Black Friday
Enterprise	\$15,000/month	15-minute response; dedicated Technical Account Manager (TAM)	Major TAMs: banks, healthcare, large e-commerce

■ What does a TAM (Technical Account Manager) do?

A TAM is your personal AWS expert — they review your architecture before Black Friday, help you choose between services, and escalate support tickets when needed. For ShopFlow at scale, a TAM would review our ECS + Aurora setup before a major product launch and suggest optimizations. Business support includes a pool of TAMs; Enterprise gives you a dedicated one.

01-17

Your Development Environment Checklist

Before starting Chapter 2, make sure your development environment is fully set up. This checklist covers everything you need to follow along with the hands-on sections in every chapter. Each item takes 5-10 minutes to install.

Tool	Purpose	Install Command / Link	Verify Installed
AWS Account	Your cloud workspace	aws.amazon.com → Create account	Log in at console.aws.amazon.com
AWS CLI v2	Run AWS commands from terminal	brew install awscli (macOS) aws.amazon.com/cli	awscli --version → aws-cli/2.x.x
Node.js 20+ (LTS)	Required by CDK	nodejs.org → LTS download	node --version → v20.x.x
AWS CDK v2	Infrastructure as code	npm install -g aws-cdk	cdk --version → 2.x.x
Java 21 (JDK)	ShopFlow app is Java 21	adoptium.net (Eclipse Temurin 21)	java --version → 21.x.x
Maven or Gradle	Java build tool	brew install maven OR brew install gradle	mvn --version or gradle --version
Docker Desktop	Build and run containers locally	docker.com/products/docker-desktop	docker --version
VS Code or IntelliJ IDE	Code editor	code.visualstudio.com or jetbrains.com/idea	Open any TypeScript or Java file
Git	Version control	brew install git git-scm.com	git --version → git 2.x.x

■ Bootstrap CDK in Your AWS Account (one-time setup)

1. After installing CDK and configuring AWS CLI, run this command once:
2. `cdk bootstrap aws://YOUR-AWS-ACCOUNT-ID/us-east-1`
3. Find your account ID: `aws sts get-caller-identity --query Account --output text`
4. CDK bootstrap creates: an S3 bucket (for CDK assets) and IAM roles (for CDK deployments)
5. You should see: ■ Environment `aws://123456789012/us-east-1` bootstrapped
6. You only need to do this once per account per region

01-18

How This Tutorial Series Works — Chapter by Chapter

Each chapter in this series follows the same structure: (1) Plain-English explanation of the AWS service, (2) Console walkthrough with step-by-step instructions, (3) CDK code to automate what you did manually, (4) How it fits into ShopFlow, (5) Best practices and interview Q&A.; You can read chapters sequentially or jump to specific services.

Chapter	Service	What You Will Build in ShopFlow
02	IAM and Security	IAM roles for each microservice; Secrets Manager for DB passwords; security audit
03	VPC and Networking	ShopFlow VPC with public/private/data subnets across 3 AZs; NAT Gateway; VPC endpoints
04	EC2 and Auto Scaling	EC2 instances for development; Auto Scaling group; Launch Template
05	S3 and CloudFront	Product image bucket; CloudFront CDN; lifecycle policies; signed URLs
06	RDS and Aurora	Aurora PostgreSQL cluster for orders/products/users; Multi-AZ; read replicas
07	DynamoDB	Shopping cart table; user sessions; On-Demand billing; Global Secondary Index
08	ElastiCache Redis	Product price cache; session store; Cluster Mode; cache-aside pattern
09	Lambda and API Gateway	Image resize function; order receipt PDF; REST API with JWT auth
10	SQS, SNS, EventBridge	Order confirmation queue; email/SMS notification fan-out; event-driven architecture
11	ECS Fargate	Deploy all ShopFlow microservices as containers; auto-scaling; blue/green deploy
12	OpenSearch	Product search index; full-text search; autocomplete; faceted navigation

Chapter	Service	What You Will Build in ShopFlow
13	Route 53 and ALB	DNS; HTTPS certificate; load balancer rules; health checks; failover routing
14	CloudWatch and X-Ray	Dashboards; alerts; distributed tracing; log analysis; SLO monitoring
15	CloudFormation and CDK	CDK best practices; Aspects; testing; CDK Pipelines; multi-stack architecture
16	CodePipeline and CI/CD	Automated build, test, and deployment pipeline; Docker; ECR; blue/green deploy
17	Cognito and WAF	User sign-up/sign-in; JWT tokens; MFA; WAF rules; bot protection
18	Kinesis and Analytics	Real-time event streaming; S3 data lake; Athena; Glue; fraud detection
19	Cost Optimization	Savings Plans; right-sizing; Reserved Instances; FinOps; \$15,000/month savings
20	High Availability and DR	Multi-region failover; Aurora Global Database; chaos engineering; DR runbook

01-19

Hands-On: Explore the AWS Console for 15 Minutes

The best way to learn the AWS Console is to click around. Here is a guided exploration exercise: visit each of the 8 services below, spend 60 seconds reading the overview page, and note what you see. Do not create anything yet — just explore. This builds familiarity before you start building ShopFlow.

Service to Explore	What to Look For in the Console	URL in Console Search
S3	The list of buckets page (empty at first); note the Region selector (S3 is global)	Type S3 in search bar
EC2	Dashboard shows running instances (0 for new accounts); note the region filter	Type EC2 URL
RDS	Databases list (empty); note options: MySQL, PostgreSQL, Aurora, SQL Server	Type RDS
Lambda	Functions list (empty); note the blue Create function button	Type Lambda
CloudWatch	Overview dashboard; note Log groups, Alarms, Metrics in left sidebar	Type CloudWatch
IAM	Users list (shows your admin user); note Users, Roles, Policies in left sidebar	Type IAM
VPC	Your VPCs list (shows the default VPC AWS creates in every new account)	Type VPC
Cost Explorer	Monthly spend chart (should show \$0 for new accounts); enable if prompted	Type Cost Explorer

■ ■ What is the Default VPC?

Every new AWS account comes with a default VPC pre-created in each region. It has public subnets in each AZ and allows EC2 instances to have public IP addresses. We will NOT use the default VPC for ShopFlow — we will create our own custom VPC in Chapter 3 with proper public/private/data subnet separation. You can safely ignore the default VPC during this tutorial.

01-BP

Best Practices and Common Beginner Mistakes

- ✓ Enable MFA on your root account AND your IAM admin user immediately after account creation — no exceptions
- ✓ Set up a \$10 billing alert before creating any resources — a forgotten NAT Gateway costs \$35/month even if unused
- ✓ Always use a named AWS Region consistently (us-east-1 for this tutorial) — resources are region-specific; an EC2 in us-east-1 does not appear when you look at us-west-2
- ✓ Run "cdk destroy" after each chapter to delete tutorial resources and stay within Free Tier limits
- ✓ Store AWS credentials (Access Key + Secret) in ~/.aws/credentials (created by "aws configure"), not in code files or environment variables
- ✓ Use the AWS CLI to verify what was created: "aws s3 ls", "aws ec2 describe-instances" — faster than clicking through the console

■ Never share your AWS Access Key or Secret Key — they give full API access to your account. If you accidentally commit them to GitHub, rotate them immediately (IAM → Users → Security credentials → Delete key → Create new key)

■ Never put AWS credentials in your application code or commit them to Git — use IAM roles (attach to ECS tasks, Lambda, EC2) which provide temporary credentials automatically

■ Do not create resources in multiple regions accidentally — check the region selector (top right) before creating anything

■ Do not use your root account for daily work — if the root account is compromised, the attacker can delete your entire AWS account

01-QR

Quick Reference and Interview Q&A

Question	Answer
What is the difference between a Region and Availability Zone?	Region = a geographic area (e.g. us-east-1 = N. Virginia). AZ = a physically separate data center within a region.
What is IAM and why not use root?	IAM (Identity and Access Management) manages who can access what in your AWS account. Never use root.
What is CDK and how is it different from CloudFormation?	CDK (Cloud Development Kit) lets you write infrastructure in TypeScript/Python/Java. CDK compiles to CloudFormation templates.
How does AWS billing work?	AWS bills you per second or per hour for most services (EC2, Fargate, RDS). Storage (S3, EBS) is billed per GB-month.
What is the Free Tier and how long does it last?	Free Tier has three types: (1) 12-month free: EC2 t2.micro 750hrs/mo, RDS db.t2.micro 750hrs/mo. (2) 30-day free: S3 5GB, Lambda 1M requests. (3) Always free: IAM, CloudWatch, SNS, SQS, ElastiCache, IAM, CloudFormation, IAM, CloudFront, IAM, CloudFront, IAM, CloudFront.